Dissertation on

"BioPathNet++ for Autoimmune Diseases: Path-Based Transfer Learning for Rare Disease Drug Repurposing"

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology
in
Computer Science & Engineering

UE23CS441A – Capstone Project Phase - 3
Project ID: PW26_PH_03

Submitted by:

TVNS Yatin	**PES1UG23CS643**
Srijan Jha	**PES1UG23AM316**
A Charan KR	**PES1UG23CS101**
Praveen Kumar Yadav	**PES1UG23CS449**

Under the guidance of

Dr. Priyanka H
Associate Professor
PES University

August – Dec, 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

(Established under Karnataka Act No. 16 of 2013)
100ft Ring Road, Bengaluru – 560 085, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled

'BioPathNet++ for Autoimmune Diseases: Path-Based Transfer Learning for Rare Disease Drug Repurposing'

is a bonafide work carried out by

TVNS Yatin	**PES1UG23CS643**
Srijan Jha	**PES1UG23AM316**
A Charan KR	**PES1UG23CS101**
Praveen Kumar Yadav	**PES1UG23CS449**

in partial fulfilment for the completion of seventh semester Capstone Project Phase - 3 (UE23CS441A) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period August – Dec, 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 7th semester academic requirements in respect of project work.

Signature **Dr. Priyanka H** Associate Professor	Signature Dr. Shylaja S S Chairperson	Signature Dr. Sridhar K S Registrar

External Viva

Name of the Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

DECLARATION

We hereby declare that the Capstone Project Phase - 3 entitled “BioPathNet++ for Autoimmune Diseases: Path-Based Transfer Learning for Rare Disease Drug Repurposing” has been carried out by us under the guidance of Dr. Priyanka H, Associate Professor, Department of Computer Science and Engineering, PES University and submitted in partial fulfilment of the course requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of PES University, Bengaluru during the academic semester August – Dec, 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES1UG23CS643	**TVNS Yatin**
PES1UG23AM316	**Srijan Jha**
PES1UG23CS101	**A Charan KR**
PES1UG23CS449	**Praveen Kumar Yadav**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Priyanka H, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS441A - Capstone Project Phase – 3.

I am grateful to the project coordinator, Dr. Chaitra B, for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr.Shylaja S S, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. Sridhar K S, Registrar, PES University for his help.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

Rare autoimmune diseases such as primary biliary cholangitis, systemic sclerosis and myasthenia gravis have few approved drugs, so drug repurposing for them has to work zero-shot: it must rank drugs for diseases whose treatments were never seen. This project builds BioPathNet++, a path-based graph neural network with a leakage-free transfer channel, trained on the PrimeKG knowledge graph on free GPUs. An audit of our earlier pipeline showed that its near-perfect scores came from test edges left in the graph, so we rebuilt the evaluation around a strict hold-out of each disease family and a full ranking of 7,957 drugs.

On 68 never-seen test diseases, an approved drug appears in the model's top 10 for 76% of diseases (best baseline 57%; MRR 0.501 versus 0.392, $p = 0.003$). Autoimmune training data helps only when data is scarce (at full scale it even lowers accuracy), and on TxGNN's own split BioPathNet++ ranks all drugs significantly better than TxGNN.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Knowledge graphs and zero-shot repurposing	1
	1.3 Motivation and how the project evolved	2
	1.4 Objectives	2
	1.5 Scope	3
	1.6 Contributions	3
	1.7 Organisation of the report	4
2	**PROBLEM STATEMENT**	**5**
	2.1 Problem definition	5
	2.2 Challenges	5
	2.3 Research questions and hypotheses	6
	2.4 Assumptions and constraints	6
3	**LITERATURE REVIEW**	**7**
	3.1 Knowledge graphs for drug repurposing	7
	3.2 Embedding-based link prediction	7
	3.3 Zero-shot and path-based methods	8
	3.4 Leakage and evaluation in machine learning	8
	3.5 Summary of related work	9
	3.6 Research gap	9
4	**PROJECT REQUIREMENTS SPECIFICATION**	**10**
	4.1 Product perspective	10
	4.2 Users and stakeholders	10
	4.3 Functional requirements	10
	4.4 Non-functional requirements	11
	4.5 Data requirements	11
	4.6 Hardware and software requirements	12
	4.7 Constraints, assumptions and risks	12
5	**SYSTEM DESIGN**	**13**
	5.1 High-level design	13
	5.2 Low-level design	15
6	**PROPOSED METHODOLOGY**	**18**
	6.1 Dataset	18
	6.2 Labels	18
	6.3 Strict-family hold-out	18

6.4 Evaluation protocols 19
6.5 Path channel 20
6.6 Memory-efficient propagation 21
6.7 Transfer channel 21
6.8 Training objective 22
6.9 Baselines 22
6.10 Evaluation metrics 22
6.11 Statistical analysis 23
6.12 Validation of unlabelled candidates 23
7 IMPLEMENTATION AND PSEUDOCODE 25
7.1 Implementation overview 25
7.2 Building the knowledge graph 25
7.3 Split generation 25
7.4 Message graph and sparse propagation 26
7.5 Forward pass and training loop 26
7.6 Evaluation 26
7.7 Path explanations 27
7.8 TxGNN head-to-head pipeline 27
7.9 Kaggle execution 27
7.10 Testing strategy 28
8 RESULTS AND DISCUSSION 29
8.1 Audit of the earlier pipeline 29
8.2 Aim 2: accuracy on unseen diseases 29
8.3 Aim 1: source of transfer 32
8.4 Aim 3: component analysis 34
8.5 Robustness 35
8.6 Head-to-head comparison with TxGNN 36
8.7 Case studies 37
8.8 External validation of new candidates 39
8.9 Discussion and limitations 40
9 CONCLUSION AND FUTURE WORK 42
9.1 Conclusion 42
9.2 Future work 42
REFERENCES 44
APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS 47
APPENDIX B: USER MANUAL 48

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Layered architecture of the BioPathNet++ system	13
5.2	Execution workflow between the laptop and Kaggle sessions	15
5.3	One training step of BioPathNet++	17
6.1	Naive hold-out versus strict-family hold-out	19
6.2	Design of the transfer study (Aim 1)	20
8.1	MRR with 95% bootstrap confidence intervals (dashed line: disease kNN)	30
8.2	AUROC versus MRR on the 68 validation diseases	32
8.3	Transfer curve: MRR on E (left) and on the six targets (right) by training condition	33
8.4	Validation MRR by connectivity tertile of the held-out disease	35
8.5	Per-target MRR for each seed; the transfer channel narrows the spread	39

LIST OF TABLES

Table No.	Title	Page No.
3.1	Comparison of related approaches with this work	9
4.1	Functional requirements	10
4.2	Non-functional requirements and how they are met	11
4.3	Hardware and software environment	12
5.1	Main modules and their responsibilities	14
5.2	Design decisions and their rationale	15
5.3	Files of the compact knowledge graph and experiment outputs	16
5.4	Main classes and functions	16
5.5	Main command-line entry points	17
6.1	Dataset and split statistics	18
7.1	Automated tests (tests/test_bpp.py)	28
8.1	Findings of the audit of the earlier (invalid) pipeline	29
8.2	Zero-shot ranking over all 7,957 drugs (targets protocol; models: mean ± sd over 3 seeds)	30
8.3	Selection-free test split (68 diseases never used for training or stopping)	31
8.4	Transfer curve: MRR [95% CI] by training condition	32
8.5	Ablations (targets protocol; p: Wilcoxon on paired validation MRR versus the full model)	34
8.6	TxGNN's autoimmune split (13 test diseases; means over 3 seeds)	36
8.7	Top-10 predictions for PBC, SSc and Sjögren syndrome (★ labelled in PrimeKG; [A-C] trial grade)	37
8.8	Top-10 predictions for AIH, MG and Crohn disease (★ labelled in PrimeKG; [A-C] trial grade)	37
8.9	Evidence for unlabelled candidates (top 20 per target, six targets)	39

CHAPTER - 1
INTRODUCTION

1.1 Background

Developing a new drug takes many years and a very large investment, and most candidates fail. Drug repurposing takes the opposite route: it looks for new therapeutic uses of drugs that are already approved. Because these drugs have known safety profiles and manufacturing processes, a successful repurposing candidate can reach patients faster and at lower cost than a new molecule [1].

The need is greatest for rare diseases. More than 7,000 rare diseases are known, yet only about 5 to 7 percent of them have an approved drug [2], and together they affect roughly 3.5 to 5.9 percent of the population [3]. Autoimmune disorders form a related area of unmet need: in a cohort of 22 million people in the United Kingdom, nineteen common autoimmune diseases together affected 10.2 percent of the population [4]. Many individual autoimmune conditions, however, are rare and have only a handful of disease-specific treatments.

This project studies six such conditions as its targets: primary biliary cholangitis (PBC), systemic sclerosis (SSc), Sjögren syndrome, autoimmune hepatitis (AIH), myasthenia gravis (MG) and Crohn disease. Three data-rich autoimmune diseases, rheumatoid arthritis (RA), systemic lupus erythematosus (SLE) and multiple sclerosis (MS), serve as possible sources of transferable knowledge.

1.2 Knowledge graphs and zero-shot repurposing

A biomedical knowledge graph (KG) represents drugs, diseases, genes and proteins, phenotypes, pathways and anatomical structures as nodes, and their relations (a drug targets a protein, a gene is associated with a disease, a drug is indicated for a disease) as typed edges. Predicting missing drug-disease edges in such a graph is a natural way to generate repurposing hypotheses [5, 6].

Most published work evaluates these predictions on randomly held-out edges. For a disease that already has several known drugs, this is easy: a model can copy the treatments of a near-identical

disease. The clinically relevant question is different: which drugs might treat a disease for which nothing is known to work? This is the *zero-shot* setting, in which whole diseases, not single edges, are hidden from training. TxGNN [2] popularised this setting, and BioPathNet [7] showed that path-based reasoning with Neural Bellman-Ford networks (NBFNet) [8] works well in it.

1.3 Motivation and how the project evolved

In the earlier phases of this capstone we built a first pipeline on an enriched version of the DRKG knowledge graph and reported test AUPRC values between 0.935 and 0.999. Before writing the final papers we audited that pipeline in detail. The audit (Chapter 8) showed that every test drug-disease pair was present as an edge of the graph the model reasoned over, and that a rule with no learning at all, based only on how many connections a drug and a disease have, scored a test AUROC of 0.972, higher than the trained model. The earlier results therefore measured leakage rather than repurposing ability.

Phase 3 rebuilt the project from the ground up on PrimeKG [9] with a leakage-controlled protocol, a memory-efficient model that fits on free GPUs, strong baselines, paired statistical tests, a head-to-head comparison with TxGNN, and five external validation layers for the model's new candidates. Every number in this report comes from the rebuilt pipeline and is generated from run logs by script.

1.4 Objectives

The project is organised around three aims:

1. **Aim 1, source of transfer.** Does knowledge transferred to rare autoimmune diseases need to come from related, data-rich autoimmune diseases, or is broad pharmacological knowledge enough?
2. **Aim 2, accuracy.** Does a path-based graph neural network rank the correct drugs higher than simple, interpretable repurposing rules on diseases it has never seen?

3. **Aim 3, components.** What do path reasoning, the transfer channel, the training loss and the background graph each contribute?

1.5 Scope

The project covers computational ranking of existing drugs for diseases, model training and evaluation on PrimeKG, comparison with baselines and with TxGNN, and evidence gathering from public sources (ClinicalTrials.gov, PubMed, regulatory approvals). It does not include wet-lab validation, clinical testing or dosing. The output is a ranked list of hypotheses for expert review, not a treatment recommendation.

1.6 Contributions

- A leakage audit of a naive repurposing pipeline, showing how test edges in the graph and random negatives produce near-perfect but meaningless scores.
- A strict-family zero-shot protocol on PrimeKG with full ranking over all 7,957 drugs and a selection-free test split.
- BioPathNet++, a BioPathNet-style NBFNet whose relation-wise sparse propagation runs on the full graph on a single 16 GB GPU, with a leakage-free transfer channel.
- A controlled transfer study with size- and label-matched control groups, answering when autoimmune training data matters.
- A head-to-head comparison with TxGNN on TxGNN's own split, including a training-coverage ablation.
- Component ablations and five validation layers for the model's unlabelled candidates.

We do not claim a new message-passing architecture: BioPathNet++ follows BioPathNet's design, and the contributions concern evaluation, transfer, efficiency and the rare autoimmune setting.

1.7 Organisation of the report

Chapter 2 defines the problem formally. Chapter 3 reviews related work. Chapter 4 specifies the requirements, Chapter 5 the high- and low-level design, and Chapter 6 the methodology. Chapter 7 describes the implementation with pseudocode and the test strategy. Chapter 8 presents and discusses the results, and Chapter 9 concludes. Appendix A lists definitions and abbreviations, and Appendix B explains how to reproduce the results.

CHAPTER - 2

PROBLEM STATEMENT

2.1 Problem definition

Let G = (V, R, E) be a knowledge graph with node set V, relation types R and edges $E \subseteq V \times R \times V$. A subset of nodes are drugs ($D \subset V$, $|D| = 7{,}957$) and a subset are diseases. For a disease u, let $P(u) \subseteq D$ be its known indications. In the zero-shot setting, a set of test diseases U is chosen and *all* drug-disease edges of every disease in U, and of every disease closely related to it, are removed from the graph used for training and inference. The task is to learn a scoring function s(u, d) from the remaining graph so that, for each held-out disease u, sorting all drugs by s(u, d) places the drugs in P(u) as high as possible.

Formally, the model must output a ranking of D for each $u \in U$; quality is measured by where the hidden true drugs land in that ranking (Section 6.10). Because there are thousands of candidate drugs and only a few correct ones per disease, the problem is a highly imbalanced ranking problem, not a balanced binary classification.

2.2 Challenges

- **Label sparsity.** The six targets have between 1 and 24 labelled drugs in PrimeKG (PBC 1, SSc 5, Sjögren 12, AIH 1, MG 7, Crohn 24). Metrics on a single target are therefore noisy and can be lower bounds.
- **Leakage.** Labels can enter the graph the model reasons over, held-out diseases can have near-duplicate subtypes in training, and random negatives make the task separable by node degree alone [10, 11].
- **Scale.** PrimeKG has about 4 million context edges; standard NBFNet implementations need memory proportional to the number of edges times the batch size, which does not fit on a free 16 GB GPU.
- **Evaluation.** AUROC against thousands of easy negatives looks excellent even for useless rankings, so the choice of metric decides whether results are meaningful [12].

2.3 Research questions and hypotheses

1. **H1 (Aim 1).** Training on related autoimmune diseases (RA, SLE, MS) transfers better to held-out autoimmune diseases than training on matched non-autoimmune diseases, and removing autoimmune diseases from full-scale training lowers accuracy on the rare targets.
2. **H2 (Aim 2).** On diseases never seen in training, BioPathNet++ ranks known drugs significantly higher than the strongest simple baseline.
3. **H3 (Aim 3).** Path reasoning and the transfer channel each contribute measurably to accuracy.

A hypothesis is accepted only if the paired bootstrap 95% confidence interval excludes zero *and* the two-sided Wilcoxon signed-rank test gives $p < 0.05$ (Section 6.11). Chapter 8 reports which hypotheses hold and which do not.

2.4 Assumptions and constraints

- PrimeKG's curated indications are treated as ground truth, although they are incomplete (some approved uses are missing; Section 8.11).
- Compute is limited to free Kaggle sessions (two NVIDIA T4 GPUs, 12-hour sessions, about 30 GPU-hours per week) and a laptop with a 4 GB GPU.
- No clinician review was available during this phase; candidate lists are prepared for later expert review.

CHAPTER - 3
LITERATURE REVIEW

This chapter reviews biomedical knowledge graphs, link-prediction methods used for repurposing, zero-shot and path-based methods, and work on leakage in machine-learning evaluation. All works cited were checked against their DOI or an exact title match before inclusion.

3.1 Knowledge graphs for drug repurposing

Hetionet integrated 29 public resources into one heterogeneous network and used metapath features to prioritise repurposing candidates [5]. The Drug Repurposing Knowledge Graph (DRKG) combined DrugBank, Hetionet, GNBR and other sources for COVID-19 work [13]. PrimeKG [9] integrates around twenty resources into 129,375 nodes and adds curated indication, contraindication and off-label edges for 17,080 diseases, with clinical descriptions; it is the graph used by both TxGNN and BioPathNet. Bonner et al. reviewed these resources and showed that results depend strongly on how the dataset is built and evaluated [6, 14].

3.2 Embedding-based link prediction

Translational and bilinear embeddings such as TransE [15], DistMult [16] and RotatE [17] learn one vector per entity and score a triple from those vectors. Relational graph convolutional networks (R-GCN) [18] add neighbourhood aggregation. These models have been applied to COVID-19 repurposing [19, 20], to rare diseases using literature-derived graphs [21], to polypharmacy side effects [22], and in systems such as KG-Predict [23] and DREAMwalk [24]. A large unified benchmark found that many reported embedding gains shrink when training and evaluation are made consistent [25]. A structural limitation matters for this project: an entity's vector is learned from its own training edges, so a disease without known drugs receives little supervision.

3.3 Zero-shot and path-based methods

3.3.1 TxGNN

TxGNN [2] combines a relational graph neural network with a metric-learning module that borrows information from similar diseases. It introduced disease-area splits of PrimeKG, in which all diseases of an area such as autoimmune or cardiovascular disease are held out together, and reported relative AUPRC gains over baselines when evaluated against sampled negatives (one per positive).

3.3.2 NBFNet and its successors

Neural Bellman-Ford networks [8] represent a (source, target) pair by generalised Bellman-Ford message passing that starts at the source node. The representation of every target summarises all paths from the source, which makes the model inductive (it needs no trained embedding for the source) and allows predictions to be explained through paths, in the spirit of subgraph reasoning [26]. A*Net [27] scales the idea with learned path pruning, and ULTRA [28] generalises it across graphs.

3.3.3 BioPathNet

BioPathNet [7] adapts NBFNet to biomedical graphs with a background regulatory graph and stringent negative sampling. On TxGNN's zero-shot splits it reports indication MRR from about 0.2 (cardiovascular) to above 0.6 (cell proliferation, adrenal gland), and Recall@20 of 0.619 against 0.539 for TxGNN on the cell-proliferation split. BioPathNet is the base design of this project.

3.4 Leakage and evaluation in machine learning

Kapoor and Narayanan identified data leakage as a main cause of irreproducible results in machine-learning based science [10]. In knowledge-graph link prediction, typical forms are test edges left in the graph, inverse or duplicate relations, and near-duplicate entities. Brière et al. benchmarked these effects for biomedical KG embeddings and found that removing train-test redundancy lowered performance and that random and cold-start splits overestimated performance on an independent

set of rare-disease indications [11]. With few positives among many negatives, precision-recall based metrics are more informative than ROC curves [12, 29].

3.5 Summary of related work

Table 3.1: Comparison of related approaches with this work

Method	Knowledge graph	Unit held out	Evaluation	Limitation for our setting
Hetionet [5]	Hetionet	Edges	Metapath classifier	Not zero-shot
KG embeddings [15, 17]	Various	Edges	Ranking of test triples	Weak for diseases without training edges
DREAMwalk [24]	Multi-layer KG	Edges	Association prediction	Not zero-shot
TxGNN [2]	PrimeKG	Disease area	AUPRC vs 1:1 sampled negatives	Easy negatives; not path-explainable
BioPathNet [7]	PrimeKG	Disease area	MRR, Hits@k over all drugs	Not studied for rare autoimmune transfer
This work	PrimeKG	Disease + its whole family	Full ranking, paired tests, 3 seeds	Single knowledge graph

3.6 Research gap

Three gaps motivate this project. First, no prior work has asked, with controlled comparisons, whether knowledge for rare autoimmune diseases is best transferred from related autoimmune diseases or from the broad graph. Second, zero-shot results are often reported under protocols that allow near-duplicate diseases or easy negatives, which can overstate performance, as our own earlier pipeline did. Third, path-based models have not been compared head-to-head with TxGNN under both TxGNN's protocol and a full ranking of all drugs. Chapters 6 to 8 address each gap.

CHAPTER - 4

PROJECT REQUIREMENTS SPECIFICATION

4.1 Product perspective

BioPathNet++ is a research software system: a Python package (bpp) with command-line tools that build the knowledge graph, generate leakage-free splits, train and evaluate models, compare them statistically and gather external evidence. It runs on a laptop for data preparation and analysis and on Kaggle GPU notebooks for training. Its outputs are result files, ranked candidate lists, figures and the numbers used in the project's papers.

4.2 Users and stakeholders

- **Researchers** who train and evaluate models and need reproducible, leakage-free results.
- **Clinical experts** who review ranked candidates; they receive a review sheet with evidence columns.
- **Reviewers and examiners** who check that every reported number can be traced to a run log.

4.3 Functional requirements

Table 4.1: Functional requirements

ID	Requirement	Implemented in
FR1	Build a compact graph from PrimeKG: nodes, context edges with explicit inverses, therapeutic pairs	build_primekg.py, kg.py
FR2	Generate zero-shot splits in which held-out disease families never contribute drug edges	splits.py, extra_splits.py
FR3	Train BioPathNet++ with switchable components (transfer channel, paths, loss, relation subset)	train.py, model.py
FR4	Resume interrupted training and stop before a session time limit	train.py, run_kaggle.py
FR5	Rank all 7,957 drugs for each evaluation disease and compute ranking metrics	metrics.py
FR6	Evaluate six baselines under the identical protocol	run_baselines.py
FR7	Compare methods with paired bootstrap confidence intervals and Wilcoxon tests	analyze.py
FR8	Generate every reported number, table and figure from result files	analyze.py, figures.py

ID	Requirement	Implemented in
FR9	Collect trial, literature, temporal and mechanism evidence for candidates, with caching	validate.py
FR10	Explain a prediction by the graph paths that support it	explain.py
FR11	Convert TxGNN's split and evaluate both models under both protocols	build_txgnn.py, txgnn_eval.py

4.4 Non-functional requirements

Table 4.2: Non-functional requirements and how they are met

Quality	Requirement	How it is met
Correctness	No held-out label may reach the model	Exclusion enforced in the graph builder; asserts and unit tests
Reproducibility	Every number traceable to a run	Fixed seeds, saved logs, cached API responses, generated paper macros
Efficiency	Train on the full graph within 16 GB of GPU memory	Sparse relation-wise propagation; measured peak 5.8 GB
Availability	Survive 12-hour session limits	Checkpoint every epoch; queue resumes from last.pt
Portability	Run on CPU, a laptop GPU or Kaggle	Pure PyTorch; device chosen at run time
Maintainability	Detect regressions quickly	Nine automated tests including a gradient check
Transparency	Report negative and borderline results honestly	Pre-defined significance rule; limitations section

4.5 Data requirements

- PrimeKG kg.csv (Harvard Dataverse, doi:10.7910/DVN/IXA7BM), about 8.1 million rows (both edge directions).
- TxGNN disease-area files (Human Disease Ontology, MONDO cross-references, grouped diseases) to define the autoimmune area.
- Public APIs for validation: ClinicalTrials.gov API v2 and NCBI PubMed E-utilities; responses are cached so that the analysis can be re-run offline.

4.6 Hardware and software requirements

Table 4.3: Hardware and software environment

Component	Specification
Training hardware	Kaggle notebook, 2 x NVIDIA T4 (16 GB each), 12-hour sessions
Development hardware	Laptop with NVIDIA RTX 3050 (4 GB); CPU for data building, baselines and analysis
Language and libraries	Python 3.11-3.13, PyTorch (sparse CSR), NumPy, SciPy, pandas, scikit-learn, matplotlib
TxGNN comparison	Separate Python 3.11 environment with PyTorch 2.4 and DGL 2.4 (CUDA 12.4)
Testing and versioning	pytest; Git and GitHub (branch bpp-v3)
Documentation	LaTeX (IEEE templates) for papers; Microsoft Word for this report

4.7 Constraints, assumptions and risks

- **Compute budget.** About 30 free GPU-hours per week; a full model run takes about 2 to 4 hours, so experiments were batched into five Kaggle sessions plus two TxGNN sessions.
- **Label incompleteness.** PrimeKG misses some approved uses (for example ustekinumab for Crohn disease), which makes some correct predictions count as errors.
- **Small evaluation sets for rare diseases.** Statistics on six targets have low power; quantitative claims therefore rest on the 68-disease validation and test sets.
- **External services.** API availability and rate limits are handled by retries and a local cache.

CHAPTER - 5

SYSTEM DESIGN

This chapter presents the high-level design (architecture, modules, data flow and key decisions) followed by the low-level design (data structures, classes, file formats and interfaces).

5.1 High-level design

5.1.1 System architecture

The system is organised in four layers (Figure 5.1). The data layer turns the raw PrimeKG file into a compact, versioned store. The protocol layer decides, for every experiment, which diseases are trained on and which are held out, and computes the leakage-free prior features. The learning layer builds the message graph and trains BioPathNet++. The evaluation and reporting layer computes metrics, runs statistics and validation, and generates the figures and numbers used in the papers.

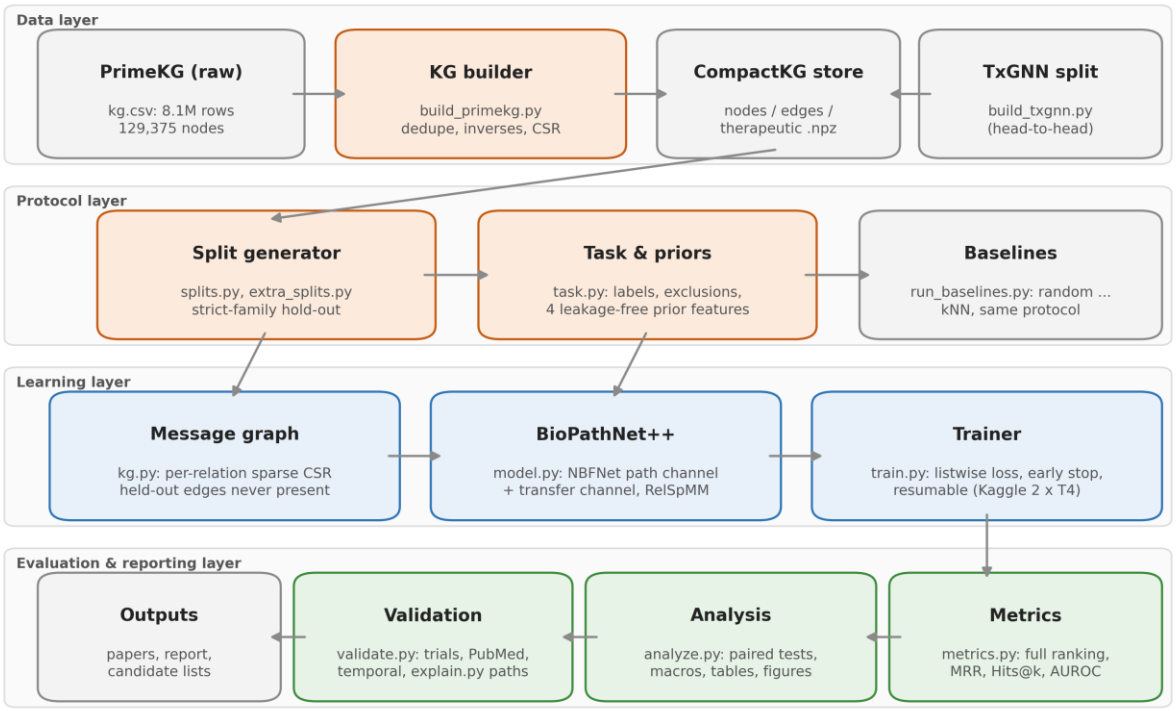


Figure 5.1: Layered architecture of the BioPathNet++ system

5.1.2 Module description

Table 5.1: Main modules and their responsibilities

Module	Responsibility	Inputs and outputs
build_primekg.py	Parse PrimeKG, deduplicate, add inverse relations, define disease families and splits	kg.csv -> CompactKG files
kg.py	CompactKG storage; MessageGraph with per-relation sparse matrices	npz/json -> CSR tensors
splits.py, extra_splits.py	Strict-family hold-out; transfer conditions; selection-free test split	graph -> splits*.json
task.py	Labels, family exclusions, prior features, baselines	split -> label matrices, features
model.py	RelSpMM, NBF layers, BioPathNet++, losses	graph, queries -> drug scores
train.py	Training loop, early stopping, checkpointing, evaluation	split -> results.json, best.pt
metrics.py	Filtered ranks, per-disease and aggregate metrics, bootstrap	scores -> metrics
run_baselines.py	Six baselines under the same protocol	split -> baselines*.json
analyze.py, figures.py	Seed averaging, paired tests, paper macros, tables, figures	run dirs -> summary.json, .tex, .pdf
validate.py, explain.py	External evidence; path explanations	scores -> CSV, review sheet, paths
build_txgnn.py, txgnn_eval.py	Convert TxGNN's split; evaluate both models under both protocols	TxGNN export -> h2h.json

5.1.3 Data flow and execution

Figure 5.2 shows how work is split between the laptop and Kaggle. The laptop builds the graph and splits, runs the unit tests and baselines, and later performs all analysis. A Kaggle notebook receives the code and the compact graph as a dataset, runs a queue of experiments with one experiment per GPU, and writes results and checkpoints. Between sessions, unfinished runs are resumed from their last checkpoint. Results are copied back, and the analysis script turns them into the numbers used in the report and papers, so no result is typed by hand.

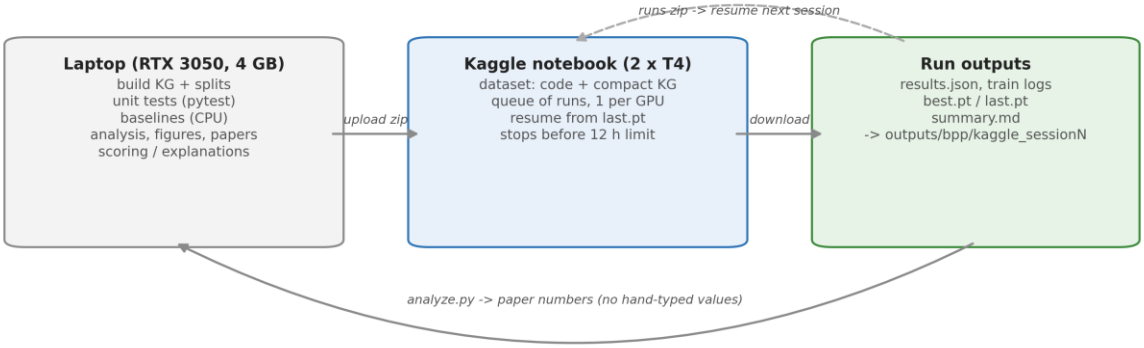


Figure 5.2: Execution workflow between the laptop and Kaggle sessions

5.1.4 Key design decisions

Table 5.2: Design decisions and their rationale

Decision	Alternative considered	Rationale
PrimeKG as the only graph	DRKG with HPO and Orphanet additions (earlier phases)	Curated drug labels, integrated phenotypes, and the benchmark graph of TxGNN and BioPathNet; the earlier additions added no edges to the target diseases
Hold out whole disease families	Hold out the disease node only	Subtypes such as Crohn's colitis otherwise leak the answer (Section 6.3)
Full ranking over all drugs	Sampled negatives	Matches the clinical use; sampled negatives let degree alone separate classes
Path-based model (NBFNet)	Entity embeddings (TransE, R-GCN)	Inductive for unseen diseases; explainable through paths
Sparse relation-wise propagation	Per-edge message tensors	Memory linear in nodes instead of edges, so the full graph fits on a T4
Listwise softmax loss	Binary cross-entropy with sampled negatives	Uses every drug as a negative, as in evaluation; better in ablation
Generated paper numbers	Manually copied numbers	Eliminates transcription errors; every value is traceable

5.2 Low-level design

5.2.1 Data structures

The compact graph is stored as NumPy arrays and JSON. Table 5.3 lists the files. Node identifiers are contiguous integers; node types and names are kept alongside. Context edges and therapeutic pairs are stored separately, so that therapeutic edges can be included or excluded per experiment.

Table 5.3: Files of the compact knowledge graph and experiment outputs

File	Content
meta.json	Node types, relation names, therapeutic relation names, statistics
nodes.npz / nodes.json	Node type of every node; node names (for reporting)
edges.npz	Context edges as (head, relation, tail) integer arrays
therapeutic.npz	Drug-disease pairs with relation (indication, contraindication, off-label)
splits*.json	Training, validation, held-out and evaluation diseases for one protocol
results.json	Per-set aggregate and per-disease metrics, best epoch, arguments
best.pt / last.pt	Best and latest model checkpoints (model, optimiser, epoch, early-stop state)
train_log.jsonl	One line per epoch: loss, validation MRR and Hits@50

5.2.2 Class design

Table 5.4: Main classes and functions

Class / function	Key members	Purpose
CompactKG (kg.py)	load(), save(), drug_ids(), disease_ids(), positives()	In-memory graph store
MessageGraph (kg.py)	build(drop_diseases)	Per-relation CSR matrices; removes a batch's drug edges
RepurposingTask (task.py)	exclusion(), eval_labels(), eval_sets(), prior_features(), baselines()	Labels, families, priors
RelSpMM (model.py)	forward(), backward()	Sparse propagation that recomputes A_r H in the backward pass
NBFLayer (model.py)	forward()	One Bellman-Ford layer: messages, aggregation, update
BioPathNetPP (model.py)	forward(graph, src, qrel, cand, prior)	Path channel + transfer channel -> scores
listwise_loss, bce_loss	-	Training objectives
Runner (train.py)	train_epoch(), evaluate(), run()	Training loop, early stopping, checkpoints
disease_metrics, aggregate	-	Filtered ranks, metrics, bootstrap CIs

5.2.3 Sequence of a training step

Figure 5.3 shows one optimisation step. A batch of eight training diseases is sampled. The drug edges of those diseases and of their families are removed from the message graph for this step, so that training mirrors the zero-shot condition. The path channel propagates from each query disease

through six layers, the transfer channel maps four prior features to a score, and the listwise loss compares the scores of the known drugs against all other drugs.

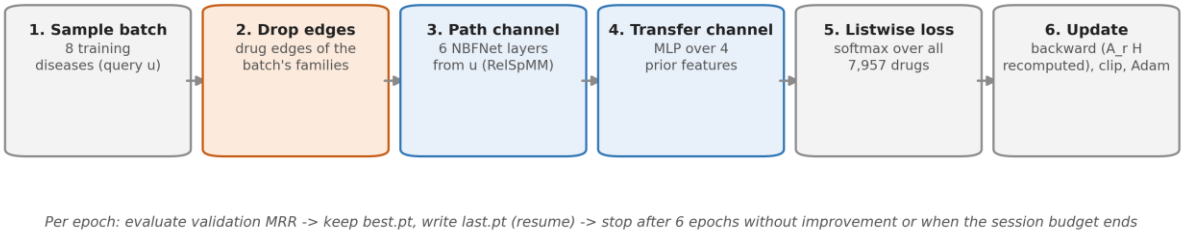


Figure 5.3: One training step of BioPathNet++

5.2.4 Command-line interfaces

Table 5.5: Main command-line entry points

Command	Purpose
python -m bpp.build_primekg --kg kg.csv.gz --txgnn-dir ... --out datasets/bpp_primekg	Build the compact graph and splits (about 35 s)
python -m bpp.run_baselines --data ... --split splits.json --out ...	Evaluate the six baselines
python -m bpp.train --data ... --split splits.json --out runs/main_s0 --prior	Train and evaluate BioPathNet++
python -m bpp.analyze; python -m bpp.figures	Generate statistics, paper numbers, tables and figures
python -m bpp.validate; python -m bpp.explain	External validation layers; path explanations
python -m pytest -q tests/test_bpp.py	Run the automated tests

CHAPTER - 6

PROPOSED METHODOLOGY

6.1 Dataset

We use PrimeKG [9]. Its 27 non-therapeutic relation types (protein-protein, drug-protein, disease-protein, disease-phenotype, tissue expression, drug-drug interaction, pathways and others) form the context graph. Every relation receives an explicit inverse, giving 54 context relations; the three therapeutic relations, also with inverses, bring the total to 60. Table 6.1 summarises the data.

Table 6.1: Dataset and split statistics

Item	Value
Nodes (drugs / diseases)	129,375 (7,957 / 17,080)
Context edges (relation types, with inverses)	4,007,433 (27, 54)
Therapeutic pairs: indication / contraindication / off-label	9,388 / 30,675 / 2,568
Targets protocol: training diseases / validation diseases (positives)	1,206 / 68 (450)
No-autoimmune protocol: training / validation / autoimmune nodes removed	1,194 / 66 / 123
Autoimmune-area test diseases (positives)	21 (147)
Transfer: training diseases in A / A' / C_E; test set E	11 / 11 / 1,185; 12
Selection-free test split: training / test diseases	1,065 / 68

6.2 Labels

The model learns to predict indications. For validation, test and area diseases the positives are their PrimeKG indications. For the six targets the positives are indications together with off-label uses, because some targets have almost no labelled indications (50 positives in total). Because these labels are so sparse, the targets are analysed as case studies, and quantitative claims rest on the larger evaluation sets.

6.3 Strict-family hold-out

Removing the drug edges of a held-out disease is not sufficient. Clinical subtypes and synonyms, linked to it by disease-disease edges, often carry the same drugs. Without further removal, all 16

Crohn indications remained reachable through "Crohn's colitis", myasthenia gravis through "adult-onset myasthenia gravis" and systemic sclerosis through "scleroderma". We therefore define the *family* of a disease as the disease together with its disease-disease neighbours and hold out whole families (Figure 6.1):

- no therapeutic edge of any held-out family member ever appears in the message graph;
- validation diseases (5% of labelled diseases, seed 0) are held out with their families, so model selection cannot reward copying from siblings;
- in every training step, the drug edges of the batch's query diseases and their families are also removed, so that training matches the zero-shot condition at test time.

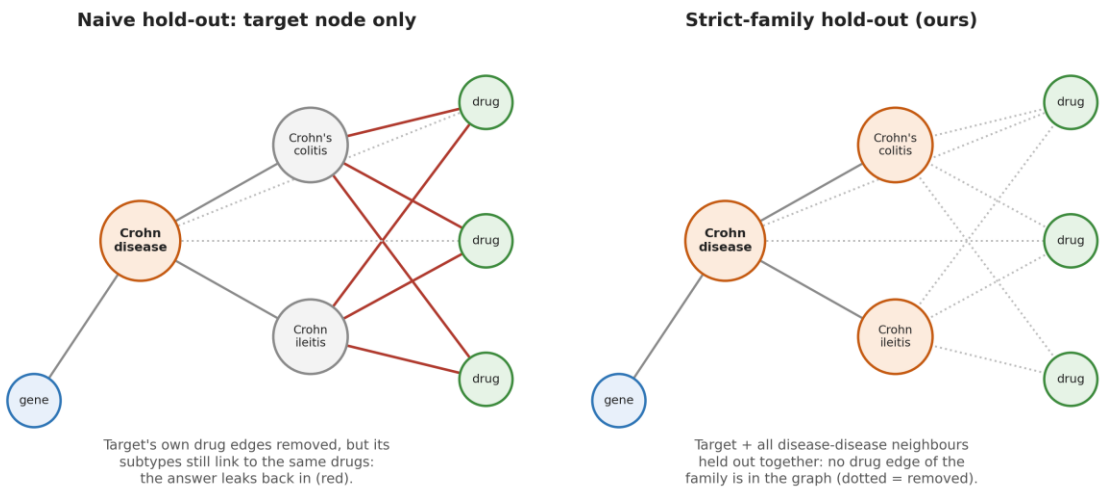


Figure 6.1: Naive hold-out versus strict-family hold-out

6.4 Evaluation protocols

- **Targets protocol.** The six target families are held out; 1,206 diseases train and 68 validation diseases select the stopping epoch.
- **No-autoimmune protocol.** Additionally removes TxGNN's autoimmune area (Disease Ontology DOID:417, 70 nodes) and the families of all nine autoimmune diseases studied

here, 123 nodes in total. Its 21 held-out area diseases with indications form a test set never used for model selection.

- **Transfer conditions (Aim 1).** A fixed test set E of 12 autoimmune-area diseases is held out in every condition; only the training diseases change (Figure 6.2). A uses the RA, SLE and MS families (11 diseases, 161 indications). A' uses three independent sets of 11 non-autoimmune diseases, each matched to A disease by disease on the number of indications (160 per set). D uses all non-autoimmune diseases, and C_E all diseases except E.
- **Selection-free test split.** 68 random non-autoimmune labelled diseases (seed 1), held out with their families, used neither for training nor for early stopping.
- **TxGNN head-to-head.** TxGNN's own autoimmune split, generated by TxGNN's code and converted so that our model trains only on edges TxGNN also trains on (Section 7.8).

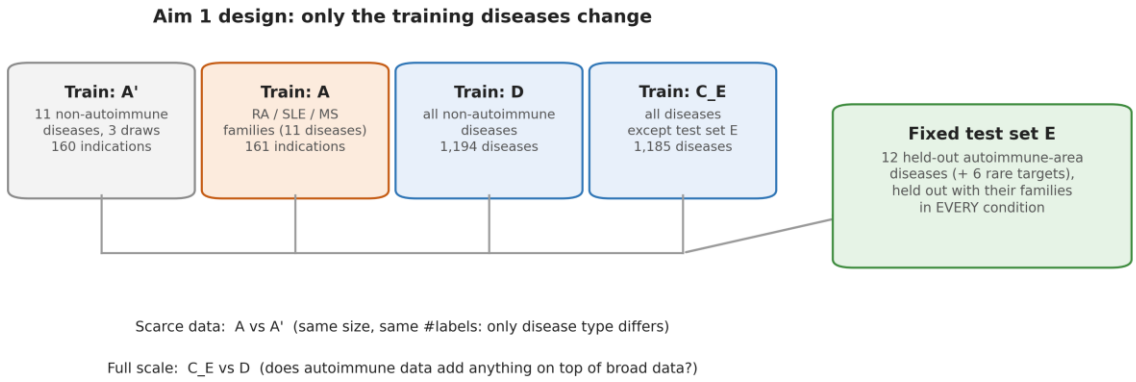


Figure 6.2: Design of the transfer study (Aim 1)

6.5 Path channel

For a query (u, r_q), where u is a disease and r_q the query relation "indication", the path channel follows NBFNet [8]. Every node v starts from an indicator boundary condition $h^{(0)}_v = 1[v = u]\cdot q$, where q is a learned embedding of r_q. Each of the L = 6 layers computes relation-specific messages and updates the node states:

$$m^{(t)}_v = (1 / \deg v) \cdot \Sigma_{(x,r,v)}\, h^{(t-1)}_x \odot w^{(t)}(r_q, r) + 1[v = u]\cdot q \qquad (6.1)$$

$$h^{(t)}_v = h^{(t-1)}_v + \mathrm{ReLU}(\mathrm{LN}(W^{(t)} [h^{(t-1)}_v ; m^{(t)}_v ; a_v \cdot m^{(t)}_v])) \quad (6.2)$$

Here $w^{(t)}(r_q, r)$ are query-dependent DistMult relation vectors [16], a_v = log(deg v + 1) / mean log(deg + 1) is a degree amplifier in the style of principal neighbourhood aggregation [30], LN is layer normalisation, and the residual connection stabilises the six-layer network. The path score of drug d is s_path(u, d) = MLP([$h^{(1)}$_d; ...; $h^{(L)}$_d; q]). Because propagation starts at u, one forward pass scores all 7,957 drugs, and no trained embedding of u is needed, so an unseen disease is handled exactly like any other node.

6.6 Memory-efficient propagation

A direct implementation materialises one message per edge, which needs memory proportional to |E|·B·d for B queries of dimension d: about 4 million edges times 8 queries times 32 dimensions per layer here. We instead write the message sum as Σ_r (A_r H) ⊙w_r over 60 sparse adjacency matrices A_r in CSR format. A custom autograd function (RelSpMM) stores only H and recomputes A_r H during the backward pass, reducing activation memory to O(|V|·B·d). The graph itself occupies 0.32 GB; training with B = 8 peaks at 5.8 GB and takes about 8.5 s per step on a T4, and the gradients pass a finite-difference check.

6.7 Transfer channel

The "++" component adds four prior features φ(u, d) for every disease-drug pair:

1. popularity: the number of training diseases for which d is indicated;
2. a disease k-nearest-neighbour score, Σ over the k = 20 most similar training diseases u' of sim(u, u')·y(u', d), where similarity is the cosine of TF-IDF weighted disease-protein and disease-phenotype profiles;
3. the number of d's protein targets that are associated with u;
4. the frequency of d among autoimmune training diseases.

Every feature is computed with u's own family excluded, identically in training and testing, so no label of a held-out disease can enter. A two-layer MLP g maps φ to a score that is added to the path score:

$$s(u, d) = s_path(u, d) + g(\varphi(u, d)) \tag{6.3}$$

6.8 Training objective

Let P(u) be the training indications of u. We minimise a multi-positive softmax cross-entropy over all candidate drugs:

$$L(u) = -(1 / |P(u)|) \cdot \Sigma_\{d \in P(u)\} \log [\exp s(u, d) / \Sigma_\{d'\} \exp s(u, d')] \tag{6.4}$$

Every drug not indicated for u acts as a negative, so no negative sampling is needed and training matches the full-ranking evaluation. The self-adversarial binary cross-entropy used by NBFNet is evaluated as an ablation.

6.9 Baselines

All baselines use the same protocol and the same family exclusions: random ranking; drug degree in the context graph; popularity (number of training diseases treated); autoimmune frequency; shared targets (number of the drug's targets associated with the disease); and disease kNN, which scores drugs by the similarity-weighted indications of the 20 most similar training diseases. Disease kNN is a transparent form of the "guilt by association" rule widely used in repurposing [24] and is the strongest baseline in our experiments.

6.10 Evaluation metrics

For a disease u with positives P(u), each positive d is ranked among all 7,957 drugs after removing the other positives (the filtered rank [15]). The main metrics are:

$$MRR(u) = (1 / |P(u)|) \cdot \Sigma_\{d \in P(u)\} \; 1 / rank_u(d) \tag{6.5}$$

$$Hits@k(u) = (1 / |P(u)|) \cdot \Sigma_\{d \in P(u)\} \; 1[rank_u(d) \leq k] \tag{6.6}$$

Both are macro-averaged over diseases. MRR originates in question-answering evaluation [31]. We also report Any@10, the share of diseases with at least one positive in the top 10, which is the easiest metric to interpret; AUROC and AUPRC over all drugs; AUROC against hard negatives (drugs indicated for other autoimmune diseases); and AUROC against degree-matched negatives. With about seven positives among 7,957 drugs, a random ranking gives MRR and AUPRC of about 0.001 and puts an approved drug in the top 10 for less than 1% of diseases. AUROC is insensitive to this imbalance [12], so it is reported but not used as a headline metric.

6.11 Statistical analysis

Models are trained with three seeds where compute allows. Per-disease metrics are averaged over seeds and then over diseases. We report seed mean and standard deviation, percentile bootstrap 95% confidence intervals over diseases (1,000 resamples) [32] and, for comparisons, the paired bootstrap confidence interval of the mean difference together with the two-sided Wilcoxon signed-rank test [33]. A difference is called significant only if both criteria agree at the 5% level; if only one agrees, it is called suggestive.

6.12 Validation of unlabelled candidates

Ranking metrics reward only drugs that PrimeKG already labels. To judge the model's new candidates we built five validation layers, comparing the model's top-20 unlabelled drugs per target with the popularity baseline's top 20 and with 20 random drugs:

- **Clinical trials:** ClinicalTrials.gov API v2; a trial counts only if it names the drug as an intervention, matched by the whole drug name. A stricter count keeps only trials that test the drug (its title names the drug, or the drug is in some but not all arms). Grades: A phase 3 or 4, B phase 2, C phase 1 or observational, D none.
- **Literature:** PubMed co-mention counts and the most relevant titles, classified as supporting, mixed, against or unrelated with AI assistance; every supporting and against label was re-checked against the abstract (animal-only studies, drugs given in every arm and inconclusive results count as mixed).

- **Temporal validation:** ranks of drugs approved for a target after the PrimeKG snapshot.
- **Mechanism:** whether a candidate's targets are among the disease's genes; pathway enrichment.
- **Path explanations:** the graph paths that most support a prediction (Section 7.6).

CHAPTER - 7

IMPLEMENTATION AND PSEUDOCODE

7.1 Implementation overview

The system is implemented as the Python package bpp (about 3,900 lines across 19 modules) with PyTorch for the model. The model has 68,226 parameters: hidden size 32, six layers, MLP hidden size 64, dropout 0.1. Training uses Adam with learning rate 5×10^{-3}, a batch of eight query diseases (151 steps per epoch), gradient clipping at 1.0, at most 20 epochs and early stopping on validation MRR with patience 5 (6 for the test-split and C_E runs). These settings were fixed in advance and not tuned. Transfer runs, which train on only 11 diseases (two steps per epoch), may run up to 60 epochs with patience 10.

7.2 Building the knowledge graph

build_primekg.py reads PrimeKG in chunks of one million rows, maps node identifiers to contiguous integers, removes duplicate edges (PrimeKG stores both directions), separates therapeutic pairs from context edges, identifies the autoimmune area through the Disease Ontology, and writes the compact store and the split files. The full build takes about 35 seconds.

7.3 Split generation

Algorithm 7.1: Strict-family split generation

```
function MAKE_SPLIT(G, targets, valid_frac, seed):
    family(x) <- {x} U disease_disease_neighbours(x)
    held_out <- U_{t in targets} family(t)
    labelled <- diseases with >= 1 indication, not in held_out
    valid <- random sample of valid_frac * |labelled| diseases (seed)
    held_out <- held_out U U_{v in valid} family(v)
    train <- labelled \ held_out
    assert family(e) disjoint from train for every evaluation disease e
    return {train, valid, held out, targets}
```

7.4 Message graph and sparse propagation

MessageGraph keeps, for each of the 60 relations, a sparse CSR matrix and its transpose. Only therapeutic edges of training diseases are ever inserted. For each training batch, build(drop_diseases) returns a view in which the drug edges of the batch's families are masked out. RelSpMM implements the propagation:

Algorithm 7.2: Relation-wise sparse propagation (RelSpMM)

```
forward(H, W, {A_r}):                          # H: |V| x B x d, W: per-relation weights
    M <- sum_r (A_r @ H) * W_r                 # sparse matrix product per relation
    save H, W for backward                     # per-edge messages are NOT stored
    return M
backward(dM):
    dH <- sum_r A_r^T @ (dM * W_r)
    dW r <- sum over nodes of (A r @ H) * dM   # A r @ H recomputed here
    return dH, dW
```

7.5 Forward pass and training loop

Algorithm 7.3: Training of BioPathNet++

```
for epoch in 1..max_epochs:
    for batch U of 8 training diseases (shuffled with seed):
        G_b <- message_graph.build(drop = families(U))      # zero-shot condition
        h <- boundary(U, q)                                  # indicator on each u
        for layer t in 1..6:  h <- NBFLayer_t(h, G_b)       # Eq. 6.1, 6.2
        s <- MLP_path(h over all drugs) + g(prior(U))       # Eq. 6.3
        loss <- listwise_loss(s, positives(U))              # Eq. 6.4
        backward; clip gradients to 1.0; Adam step
    mrr <- evaluate(validation diseases)
    if mrr improved: save best.pt else bad <- bad + 1
    save last.pt                                             # allows resuming
    if bad >= patience or session budget exhausted: stop
load best.pt; evaluate all sets; write results.json
```

7.6 Evaluation

Algorithm 7.4: Full-ranking evaluation of one disease

```
scores <- model(u) over all 7,957 drugs
for each positive d in P(u):
    rank(d) <- 1 + #{drugs d' not in P(u) with score(d') > score(d)}   # filtered rank
MRR(u) <- mean(1 / rank(d));  Hits@k(u) <- mean(rank(d) <= k)
AUROC, AUPRC over all drugs; AUROC vs hard and degree-matched negatives
aggregate: mean over diseases; bootstrap CI over diseases
```

7.7 Path explanations

explain.py re-runs the forward pass while keeping every layer's node states, computes the importance of each edge as gradient times input of its message, keeps the most important edges and runs a beam search for the highest-scoring paths from the disease to the drug of up to four hops [8]. A check confirms that this replica of the forward pass reproduces the model's scores exactly.

7.8 TxGNN head-to-head pipeline

TxGNN depends on the DGL library, which has no builds for the Python version of current Kaggle images. The Kaggle notebook (run_txgnn.py) therefore creates a separate Python 3.11 environment with PyTorch 2.4 and DGL 2.4, downloads TxGNN's code at a pinned commit, and generates TxGNN's autoimmune split with TxGNN's own code (with a fixed random seed, because TxGNN draws part of its test set without one). build_txgnn.py converts the split so that our model never sees an edge TxGNN does not train on: TxGNN's random test edges of other diseases are removed from our graph and labels, and the area diseases are held out with their families. Both models are then trained with three seeds and scored by txgnn_eval.py under TxGNN's protocol and under full ranking. The whole pipeline was first rehearsed on a small real-ID extract of PrimeKG on the laptop before running on Kaggle.

7.9 Kaggle execution

run_kaggle.py runs inside a Kaggle notebook. It locates the code and data in the attached dataset, restores unfinished runs from earlier sessions, runs the baselines and a smoke test, and then executes a queue of experiments with one worker thread per GPU. Each run receives the time left in the session as its budget, so that no run can outlive Kaggle's 12-hour limit and lose the session's output. Five sessions were needed for the main experiments and two for the TxGNN comparison.

7.10 Testing strategy

Testing combines automated unit tests, built-in assertions, reproducibility checks and statistical testing. Table 7.1 lists the automated tests, all of which pass.

Table 7.1: Automated tests (tests/test_bpp.py)

Test	What it verifies
RelSpMM matches dense and passes gradcheck	Sparse propagation equals dense computation; gradients are correct
Batch diseases lose their drug edges	Edges of a batch's families are absent from the propagation matrices
Inverse edges reach diseases without out-edges	Every disease is reachable (fixes the old one-directional bug)
Model forward pass learns a toy task	End-to-end training reduces loss and ranks positives first
Extra splits never train on evaluation families	No overlap between training and evaluation families in any split
Explanation replica matches the model	Path-explanation forward pass equals the model's scores
Validation helpers	Trial grading, name cleaning and evidence parsing
TxGNN conversion and protocol	Converter output, identifier mapping and TxGNN-protocol metrics
TxGNN full-coverage conversion	No test pair of any disease reaches a training disease's table

Beyond unit tests, the pipeline asserts leakage rules at run time (for example, the TxGNN converter refuses to write a split in which a test pair is visible to a training disease). Results downloaded from Kaggle were verified before use: the code on Kaggle was compared with the repository, resumed runs were checked against their logs, and the TxGNN evaluation was recomputed locally from the raw predictions. Our re-implementation of TxGNN's evaluation reproduced TxGNN's own evaluation output exactly.

CHAPTER - 8
RESULTS AND DISCUSSION

This chapter first summarises the audit that motivated the rebuild, then answers the three aims, reports robustness checks and the TxGNN comparison, and ends with case studies, external validation and a discussion of limitations.

8.1 Audit of the earlier pipeline

The earlier pipeline reported test AUPRC between 0.935 and 0.999. Table 8.1 lists what the audit found.

Table 8.1: Findings of the audit of the earlier (invalid) pipeline

Finding	Value
Test positive pairs present as edges of the message graph	277 of 277
Test AUROC of a no-learning rule log deg(drug) + log deg(disease)	0.972 (model: 0.941)
Targets unreachable by the subgraph extractor (score independent of the disease)	5 of 6
Positive labels that are text-mined (GNBR) edges	90-91%
Spurious relation types created by the phenotype parser	2,325

Three mechanisms explain the high scores. Every test pair was an edge the model could traverse (target leakage). Negatives were random drug-disease pairs, which a degree count separates easily, so the no-learning rule beat the model. Neighbourhoods were extracted along incoming edges only, so five of the six targets were unreachable and the same drug ranked first for every disease. Some labels were also wrong: thalidomide for PBC is a text-mined edge contradicted by a negative randomised trial [34]. All results below come from the rebuilt pipeline.

8.2 Aim 2: accuracy on unseen diseases

Table 8.2 compares the model with the six baselines on the 68 validation diseases and the six targets, and Figure 8.1 shows MRR with confidence intervals. On the validation diseases BioPathNet++ reaches MRR 0.394 ± 0.003 over three seeds, against 0.281 for disease kNN and 0.046 for popularity. The paired gain over disease kNN is +0.113 (95% CI +0.056 to +0.179;

Wilcoxon $p < 0.001$); the model is better on 43 diseases, tied on 13 and worse on 12. Against hard negatives, the drugs of other autoimmune diseases, its AUROC is 0.799 against 0.661 for kNN and 0.476, below chance, for popularity, so the model does more than rank popular drugs.

Table 8.2: Zero-shot ranking over all 7,957 drugs (targets protocol; models: mean ± sd over 3 seeds)

Method	Valid MRR	Valid H@10	Valid AUROC-hard	Targets MRR	Targets H@10
Random	0.001	0.002	0.488	0.001	0.000
Drug degree	0.004	0.003	0.421	0.003	0.000
Popularity	0.046	0.049	0.476	0.099	0.148
Autoimmune frequency	0.038	0.040	0.128	0.094	0.169
Shared targets	0.057	0.105	0.530	0.008	0.000
Disease kNN	0.281	0.350	0.661	0.296	0.424
Prior only (transfer channel)	0.306 ± 0.003	0.354 ± 0.006	0.700 ± 0.013	0.330 ± 0.025	0.490 ± 0.007
Paths only (NBFNet)	0.405 ± 0.013	0.479 ± 0.025	0.829 ± 0.011	0.463 ± 0.087	0.657 ± 0.025
BioPathNet++ (paths + prior)	0.394 ± 0.003	0.456 ± 0.018	0.799 ± 0.010	0.475 ± 0.017	0.648 ± 0.033

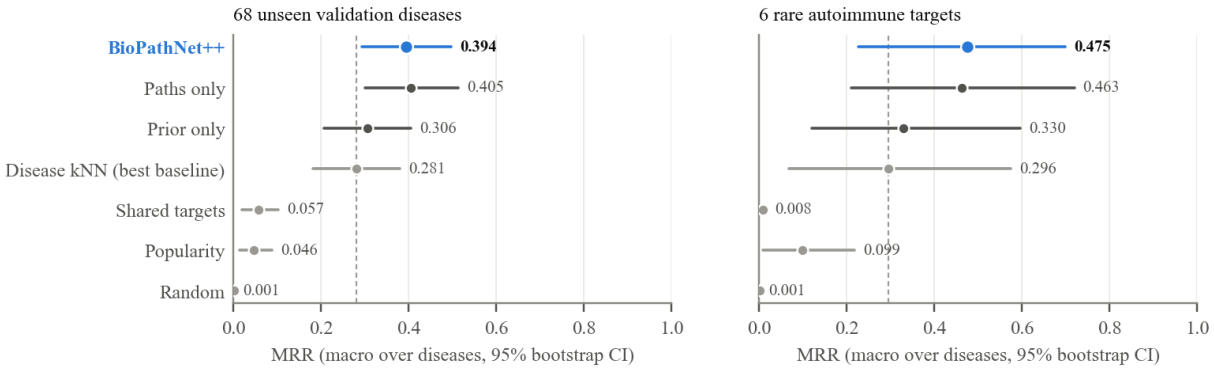


Figure 8.1: MRR with 95% bootstrap confidence intervals (dashed line: disease kNN)

On the six targets MRR is 0.475 against 0.296 for kNN. The difference (+0.179, CI −0.027 to +0.370) points the same way for four of the six targets, but with six diseases it is not significant ($p = 0.188$), and we do not claim it.

8.2.1 Selection-free test split

Because the validation diseases also choose the stopping epoch, their scores are slightly optimistic. The 68 test diseases were never used for training or stopping (Table 8.3). There, an approved drug appears in the model's top 10 for 76% of diseases, against 57% for disease kNN, 19% for popularity and under 1% for a random ranking. MRR is 0.501 against 0.392 (paired difference +0.109, CI +0.044 to +0.180, p = 0.003; three seeds), and AUPRC is 0.556 against a random level of about 0.001. The median rank of a known drug is 2, and 43% of known drugs are ranked first. The test diseases are non-autoimmune and easier for every method (kNN also rises from 0.281 to 0.392), so the two levels are not directly comparable; what matters is that the gain holds on diseases no modelling choice has seen. H2 is therefore supported.

Table 8.3: Selection-free test split (68 diseases never used for training or stopping)

Method	Any@10	MRR	H@10	AUROC-hard	AUPRC	Mean rank
Random	0.8%	0.001	0.005	0.471	0.002	4,349
Popularity	19%	0.062	0.070	0.562	0.084	918
Disease kNN	57%	0.392	0.459	0.721	0.429	742
BioPathNet++ (3 seeds)	76%	0.501	0.637	0.862	0.556	329

8.2.2 Why AUROC is not the headline metric

Figure 8.2 plots AUROC against MRR. Baselines without any learning reach AUROC between 0.78 and 0.86, yet their MRR stays below 0.06. With thousands of negatives per positive, AUROC rewards placing a positive above the long tail of irrelevant drugs, not placing it near the top of the list, which is what a clinician would read. The same effect let the degree rule reach AUROC 0.972 in the audit.

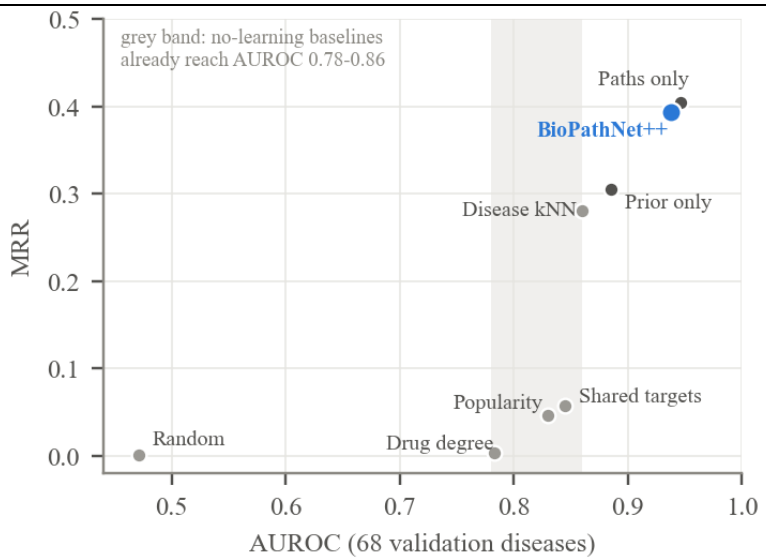


Figure 8.2: AUROC versus MRR on the 68 validation diseases

8.3 Aim 1: source of transfer

Figure 8.3 and Table 8.4 give the transfer curve on the fixed test set E of 12 autoimmune-area diseases.

Table 8.4: Transfer curve: MRR [95% CI] by training condition

Training data	Diseases	E: BioPathNet++	E: disease kNN	Targets: BioPathNet++
A': matched non-autoimmune (3 draws)	11	0.126 [0.04, 0.22]	0.086 [0.02, 0.17]	0.041 [0.00, 0.08]
A: RA / SLE / MS families	11	0.290 [0.09, 0.50]	0.258 [0.06, 0.47]	0.124 [0.01, 0.26]
D: all non-autoimmune	1,194	0.532 [0.34, 0.72]	0.318 [0.13, 0.53]	0.487 [0.20, 0.73]
C_E: all except E	1,185	0.410 [0.23, 0.62]	0.271 [0.08, 0.50]	0.539 [0.26, 0.77]

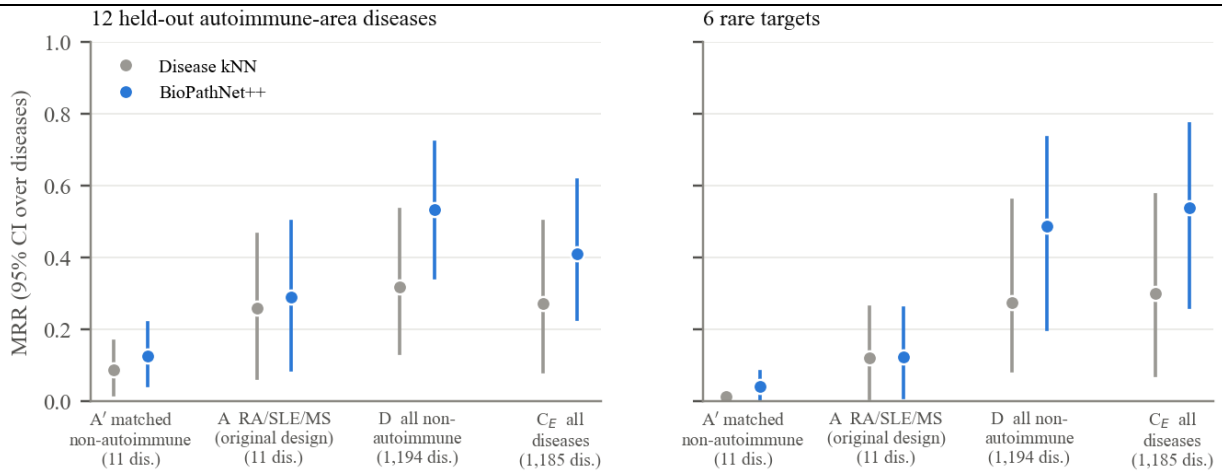


Figure 8.3: Transfer curve: MRR on E (left) and on the six targets (right) by training condition

Scarce data. Trained only on the RA, SLE and MS families (A), the model reaches MRR 0.290 on E, against 0.126 when trained on matched non-autoimmune diseases (A'). The paired difference, +0.164 (CI +0.037 to +0.316; p = 0.016; better on 9 of 12 diseases), is significant, and it also holds with each run's final checkpoint instead of the early-stopped one (+0.146, p = 0.021). Disease kNN shows the same pattern, and in this regime the model is only slightly ahead of it. The A-trained model ranks the same corticosteroids first for every disease: with 11 training diseases, what transfers is essentially a drug-class prior. A paths-only repetition confirms this: without the transfer channel both A and A' stay near the floor (MRR 0.007 and 0.010), so the advantage of A comes from the transfer channel, not from path reasoning.

Full scale. With broad training data and no autoimmune disease (D), accuracy on E rises to 0.532. Adding the autoimmune diseases (C_E, three seeds) does not help but lowers it to 0.410 (difference −0.122, CI −0.200 to −0.052, p = 0.010; lower on 9 of 12 diseases). Both conditions stop on the same validation diseases, and disease kNN drops in the same way (0.318 to 0.271), so the drop comes from which diseases are in the training set, not from the model or from early stopping. On the six targets, removing all 123 autoimmune nodes from training leaves MRR unchanged (0.487 versus 0.475, three seeds each, p = 0.812). On the 21 autoimmune-area diseases, never used for selection, the no-autoimmune model reaches 0.449 against 0.311 for kNN (difference +0.138, CI +0.010 to +0.280, p = 0.147), which by our rule is suggestive rather than significant.

H1 is therefore only partly supported. Related autoimmune diseases are the most useful source of knowledge when training data is scarce, but at full scale the broad pharmacological graph is the better source, and adding autoimmune training labels even lowers accuracy on held-out autoimmune diseases.

8.4 Aim 3: component analysis

Table 8.5: Ablations (targets protocol; p: Wilcoxon on paired validation MRR versus the full model)

Variant	Valid MRR	Valid H@10	Targets MRR	p
Full model (3 seeds)	0.394 ± 0.003	0.456 ± 0.018	0.475 ± 0.017	-
Without transfer channel (3 seeds)	0.405 ± 0.013	0.479 ± 0.025	0.463 ± 0.087	0.148
Without path reasoning (prior only, 3 seeds)	0.306 ± 0.003	0.354 ± 0.006	0.330 ± 0.025	< 0.001
Background relation subset (72% fewer edges, 3 seeds)	0.404 ± 0.006	0.492 ± 0.004	0.504 ± 0.075	0.299
BCE loss instead of listwise (3 seeds)	0.378 ± 0.028	0.452 ± 0.017	0.320 ± 0.046	0.344

Table 8.5 isolates the components. Removing the transfer channel leaves validation MRR statistically unchanged (0.405, $p = 0.148$), while removing path reasoning drops it to 0.306 ($p < 0.001$), close to disease kNN, which is one of its inputs. Path reasoning is therefore the source of the accuracy. The transfer channel does not raise the mean but makes the model much more stable across seeds on the targets: the standard deviation of target MRR falls from 0.087 to 0.017 (Figure 8.5), which matters when individual rankings are used as case studies. An interim claim that the channel improved rare-target accuracy, based on one seed, did not survive replication and was withdrawn.

With binary cross-entropy instead of the listwise softmax, validation MRR is 0.378, not significantly lower (difference +0.016, CI −0.005 to +0.039, $p = 0.344$, three seeds each). A one-seed run had suggested a clear gap (0.347, $p = 0.005$), but that seed stopped early at epoch 9, while the two new seeds kept improving to the 20-epoch cap. BCE is lower on the six targets (0.320 against 0.475; too few to test), and the listwise loss is kept because it needs no negative sampling. Dropping tissue-expression and drug-drug edges, as in BioPathNet's background graph, shrinks the context graph from 4,007,433 to 1,133,029 edges (72% fewer) with no loss in accuracy (0.404, p =

0.299, three seeds), 11% faster steps and lower memory. Most of PrimeKG's edges are therefore redundant for this task. H3 is supported for path reasoning, but not for the loss or for the transfer channel's effect on the mean.

8.5 Robustness

8.5.1 Selection bias

From the training logs, the gap between best-epoch and last-epoch validation MRR is 0.002 to 0.016 for the full model and 0.022 to 0.030 for paths only. This is an order of magnitude smaller than the gain over disease kNN, but larger than the difference between the full model and paths only, which is therefore a tie. The selection-free test split confirms the main conclusion.

8.5.2 Sparsity

Rare diseases are sparsely annotated, so we stratified the validation diseases by their number of context edges (Figure 8.4). Accuracy grows with connectivity (Spearman $\rho = 0.30$, $p = 0.012$), and BioPathNet++ is ahead of disease kNN in every tertile: 0.236 against 0.160 for the sparsest third, 0.389 against 0.218 for the middle third and 0.557 against 0.462 for the densest third.

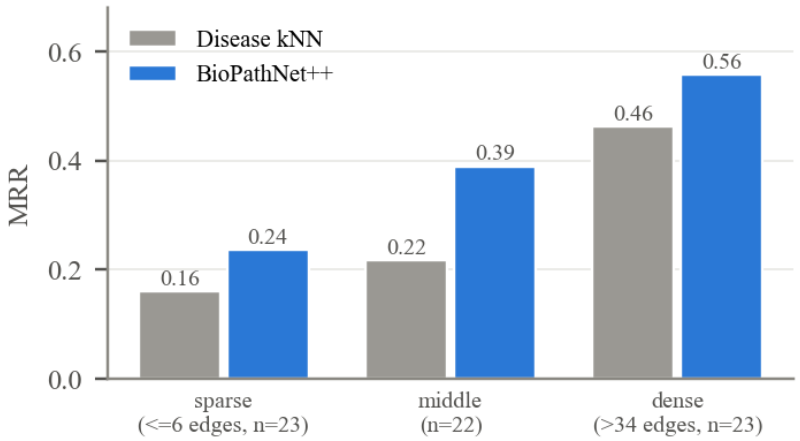


Figure 8.4: Validation MRR by connectivity tertile of the held-out disease

8.6 Head-to-head comparison with TxGNN

Both models were run on TxGNN's own autoimmune disease-area split, generated by TxGNN's code, with three seeds each. The test set is the 13 held-out area diseases that have indications (75 drug-disease pairs). Our model trained on 1,139 diseases (plus 63 for early stopping) and only on edges that TxGNN also trains on; TxGNN learns from the training edges of all 1,299 non-test diseases, so our model still sees less data. TxGNN was trained with the settings of its reproduction script (one full pre-training epoch of about 1.2 hours on a T4 and 500 fine-tuning epochs). Table 8.6 gives the results.

Table 8.6: TxGNN's autoimmune split (13 test diseases; means over 3 seeds)

Method	R@100	AUROC	AUPRC	MRR	H@100	AUROC
	TxGNN protocol			Full ranking		
TxGNN, published (5 seeds)	0.338	-	-	-	-	-
TxGNN (our run)	0.536	0.901	0.161	0.048	0.232	0.753
BioPathNet++	0.614	0.777	0.343	0.273	0.618	0.936
Ablation, strict coverage (1 seed): TxGNN	0.474	0.909	0.147	0.026	0.155	0.736
Ablation, strict coverage (1 seed): BioPathNet++	0.475	0.746	0.101	0.061	0.398	0.910

When all 7,957 drugs are ranked, BioPathNet++ is significantly better: MRR 0.273 against 0.048 (difference +0.226, CI +0.055 to +0.422, $p = 0.040$), Hits@100 0.618 against 0.232 ($p = 0.002$) and AUROC 0.936 against 0.753 ($p < 0.001$, better on all 13 diseases). Under TxGNN's own protocol, which ranks only the 1,798 drugs that have at least one indication, no difference is significant: BioPathNet++ has the higher Recall@100 (0.614 against 0.536) and AUPRC (0.343 against 0.161), and TxGNN the higher AUROC (0.901 against 0.777; CI excludes zero but $p = 0.168$). The ranked lists explain why the protocols disagree: only 23% of TxGNN's top-100 drugs have an indication for any disease, against 99% for BioPathNet++. TxGNN places many drugs with no indication near the top, and its protocol removes them before scoring.

The evaluation code was checked carefully: our implementation of TxGNN's protocol reproduces TxGNN's own evaluation exactly on every seed. A first run with a stricter conversion, in which our model trained on only 637 diseases, serves as an ablation: keeping all eligible diseases in training

raised our model's full-ranking MRR by +0.213 (p = 0.013), while TxGNN's AUROC lead under its own protocol shrank only from −0.163 to −0.124. The comparison rests on one split and 13 diseases from one disease area.

8.7 Case studies

Table 8.7 and Table 8.8 list the ten highest-ranked drugs for each target (rank averaged over three seeds). A star marks drugs that PrimeKG already labels for the disease; the letter in brackets is the strongest registered trial that tests the drug in that disease (A phase 3 or 4, B phase 2, C phase 1 or observational); trials that use the drug only as background therapy do not count.

Table 8.7: Top-10 predictions for PBC, SSc and Sjögren syndrome (★ labelled in PrimeKG; [A-C] trial grade)

Rank	PBC	SSc	Sjögren
1	Ursodeoxycholic acid★ [A]	Benzoic acid★	Propylene glycol★
2	Cholic acid	Aminobenzoic acid★	Lifitegrast
3	Doripenem	Prednisolone [B]	Povidone★
4	Homatropine	Triamcinolone	Povidone K30★
5	Ampicillin	Betamethasone	Prednisolone★
6	Dehydrocholic acid	Methylprednisolone [C]	Lutein
7	Metacycline	Bosentan★ [A]	Polyethylene glycol 400★
8	Prednisolone	Prednisone	Polyvinyl alcohol★
9	Chenodeoxycholic acid	Ampicillin	Polyethylene glycol★
10	Triamcinolone	Cortisone acetate	Polysorbate 80★

Table 8.8: Top-10 predictions for AIH, MG and Crohn disease (★ labelled in PrimeKG; [A-C] trial grade)

Rank	AIH	MG	Crohn
1	Prednisolone	Ambenonium★	Hydrocortisone★
2	Triamcinolone	Neostigmine★ [C]	Cortisone acetate★
3	Hydrocortisone acetate	Pyridostigmine★ [A]	Triamcinolone★
4	Sulfacetamide	Distigmine★	Prednisolone★ [A]
5	Hydrocortisone	Prednisolone★	Ofloxacin
6	Betamethasone	Triamcinolone	Ciprofloxacin [A]
7	Methylprednisolone [A]	Amifampridine [A]	Norfloxacin
8	Cortisone acetate	Guanidine	Cefuroxime
9	Prednisone [A]	Betamethasone	Betamethasone★

Rank	AIH	MG	Crohn
10	Acetarsol	Hydrocortisone	Prednisone★ [A]

- **PBC.** Ursodeoxycholic acid, the first-line therapy and the only labelled drug, ranks first in all seeds, followed by other bile acids.
- **Myasthenia gravis.** The four labelled cholinesterase inhibitors rank 1 to 4 in all seeds. Amifampridine, approved for Lambert-Eaton syndrome, is a new candidate with a completed phase-3 trial in MG (NCT03304054); it helped in MuSK-antibody MG [35] but not, in a 2026 trial, in acetylcholine-receptor-antibody MG [36].
- **Systemic sclerosis.** The labelled drugs benzoic acid and aminobenzoic acid lead, followed by corticosteroids; the labelled endothelin antagonist bosentan ranks eighth. High-dose corticosteroids are linked to scleroderma renal crisis, so these are candidates a clinician would question.
- **Sjögren syndrome.** Labelled tear-substitute ingredients (propylene glycol, povidone, polyethylene glycols) and prednisolone lead; lifitegrast, a dry-eye drug, is the main new candidate, supported only by a mouse dry-eye study and with no registered trial in Sjögren syndrome.
- **Crohn disease.** Labelled corticosteroids rank 1 to 4; ciprofloxacin, which has registered trials in Crohn disease (grade A), follows, together with other antibiotics.
- **Autoimmune hepatitis.** The model fails by the metric but arguably not clinically: PrimeKG lists one off-label drug, while the model ranks prednisolone, a standard therapy and the comparator in AIH trials (prednisone in NCT00838214), first. This is a label gap.

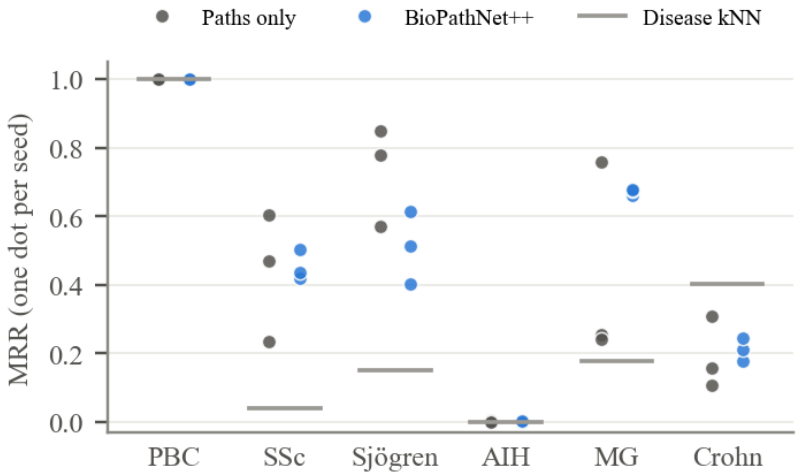


Figure 8.5: Per-target MRR for each seed; the transfer channel narrows the spread

8.8 External validation of new candidates

Table 8.9: Evidence for unlabelled candidates (top 20 per target, six targets)

Evidence	BioPathNet++ (n = 86)	Popularity (n = 120)	Random (n = 120)
Trial names the drug, grade A-C	17%	14%	0%
Trial names the drug, phase 2 or higher	15%	12%	0%
Trial tests the drug, grade A-C	13%	8%	0%
Trial tests the drug, phase 2 or higher	12%	8%	0%
Any PubMed co-mention	63%	72%	18%
PubMed: evidence of benefit	15%	10%	0%
PubMed: harm or no benefit	5%	12%	0%
At least one drug target among disease genes	17%	16%	3%

Table 8.9 summarises the evidence. Counting any trial that names the drug, 17% of the model's unlabelled candidates have a registered trial for the disease, against 14% for popularity and 0% for random drugs. Many such trials give a corticosteroid only as background therapy, premedication or transplant conditioning, so we also count only trials that test the drug: 13% for the model (9 at phase 3 or higher), 8% for popularity and 0% for random drugs. Examples are amifampridine for MG, ciprofloxacin for Crohn disease, N-acetylcysteine for SSc, and ustekinumab for Crohn disease (approved for Crohn disease in 2016 but not labelled in PrimeKG). A trial is not proof of benefit,

as the two amifampridine trials show. After the benefit and harm labels were checked against the abstracts, PubMed shows benefit for 15% of the model's candidates against 10% for popularity, and harm or no benefit for 5% against 12%, but popular drugs are co-mentioned more often. The evidence therefore shows that the candidates are plausible, but it does not show that they are clearly better than popular drugs; this is the main weakness of the repurposing claim.

In the temporal check, six drugs approved for the targets after the PrimeKG snapshot rank at a median of 1,104 of 7,957 (top 14%); five of six beat the random median, but with six drugs this is not significant ($p = 0.11$). Risankizumab, approved for Crohn disease in 2022, ranks 37th. Path explanations recover sensible mechanisms (cholinesterase inhibitors reach MG through the MuSK gene and neuromuscular-junction disease; ursodeoxycholic acid reaches PBC through biliary diseases) and also expose weak ones (ciprofloxacin reaches Crohn disease through "mouth disease" and sinusitis).

8.9 Discussion and limitations

Against chance and simple rules the model is a clear improvement, and its MRR values fall within the range BioPathNet reports on TxGNN's disease areas [7], although the protocols differ. The earlier pipeline's values near 0.99 measured an easier and different quantity, separating known pairs that were already in the graph from random pairs, and should be read as a warning about near-perfect zero-shot numbers. The main limitations are:

- the targets have 1 to 24 labelled drugs, so per-target metrics are noisy and can be lower bounds;
- the model favours corticosteroids (41% of top-20 slots) and antibacterials (21%), partly a popularity bias;
- the TxGNN comparison covers 13 diseases of one split, and BioPathNet itself was not run on a shared split;
- the temporal test covers only six drugs;

- literature labels were assigned with AI assistance and checked against abstracts, but no clinician has reviewed them;
- no clinician has reviewed the candidates yet; the rankings are research hypotheses, not recommendations.

CHAPTER - 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project set out to rank existing drugs for rare autoimmune diseases whose treatments a model has never seen. An audit of our earlier pipeline showed that its near-perfect results came from leakage and easy negatives, so we rebuilt the system around a strict-family zero-shot protocol, full ranking over all 7,957 drugs, strong baselines and pre-defined statistical tests.

The rebuilt BioPathNet++ answers the three aims as follows. On accuracy (Aim 2), it outperforms every baseline evaluated on the same split: on 68 never-seen test diseases, an approved drug is in its top 10 for 76% of diseases against 57% for the best baseline. On the source of transfer (Aim 1), related autoimmune diseases are the most useful knowledge when training data is scarce, but at full scale the broad graph is the better source and adding autoimmune training data even lowers accuracy. On components (Aim 3), path reasoning carries the accuracy, the transfer channel adds stability across seeds, the choice of loss makes no significant difference, and 72% of the graph's edges can be removed without loss. On TxGNN's own split, the model ranks all drugs significantly better than TxGNN and is not significantly worse under TxGNN's protocol.

The work is useful in three ways: as a tool that produces evidence-annotated candidate lists for expert review, as a reproducible and leakage-controlled evaluation protocol that other groups can reuse, and as a practical demonstration that full-graph path reasoning fits on free GPUs. The results are written up as an IEEE conference paper and a journal manuscript.

9.2 Future work

- Clinician review of the top candidates and of the literature labels with the prepared review sheet.
- Knowledge-graph embedding baselines and BioPathNet's official code on the same split.
- The TxGNN comparison on further disease areas, to move beyond 13 test diseases.

- Reducing the corticosteroid bias, for example by popularity-adjusted ranking, and reporting less obvious candidates separately.
- Adding curated, non-text-mined relations from other databases as background context, tested as an ablation.

REFERENCES

[1] S. Pushpakom et al., "Drug repurposing: progress, challenges and recommendations," Nature Reviews Drug Discovery, vol. 18, no. 1, pp. 41–58, 2019, doi: 10.1038/nrd.2018.168.

[2] K. Huang et al., "A foundation model for clinician-centered drug repurposing," Nature Medicine, vol. 30, no. 12, pp. 3601–3613, 2024, doi: 10.1038/s41591-024-03233-x.

[3] S. Nguengang Wakap et al., "Estimating cumulative point prevalence of rare diseases: analysis of the Orphanet database," European Journal of Human Genetics, vol. 28, no. 2, pp. 165–173, 2020, doi: 10.1038/s41431-019-0508-0.

[4] N. Conrad et al., "Incidence, prevalence, and co-occurrence of autoimmune disorders over time and by age, sex, and socioeconomic status: a population-based cohort study of 22 million individuals in the UK," The Lancet, vol. 401, no. 10391, pp. 1878–1890, 2023, doi: 10.1016/S0140-6736(23)00457-9.

[5] D. S. Himmelstein et al., "Systematic integration of biomedical knowledge prioritizes drugs for repurposing," eLife, vol. 6, p. e26726, 2017, doi: 10.7554/eLife.26726.

[6] S. Bonner et al., "A review of biomedical datasets relating to drug discovery: a knowledge graph perspective," Briefings in Bioinformatics, vol. 23, no. 6, p. bbac404, 2022, doi: 10.1093/bib/bbac404.

[7] E. Y. Hu et al., "Enhancing link prediction in biomedical knowledge graphs with BioPathNet," Nature Biomedical Engineering, vol. 10, no. 8, pp. 1747–1769, 2026, doi: 10.1038/s41551-025-01598-z.

[8] Z. Zhu, Z. Zhang, L.-P. Xhonneux and J. Tang, "Neural Bellman-Ford networks: A general graph neural network framework for link prediction," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[9] P. Chandak, K. Huang and M. Zitnik, "Building a knowledge graph to enable precision medicine," Scientific Data, vol. 10, p. 67, 2023, doi: 10.1038/s41597-023-01960-3.

[10] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, p. 100804, 2023, doi: 10.1016/j.patter.2023.100804.

[11] G. Brière, T. Stosskopf, B. Loire and A. Baudot, "Benchmarking the impact of data leakage on the performance of knowledge graph embedding models for biomedical link prediction," Bioinformatics, vol. 42, no. 8, p. btag608, 2026, doi: 10.1093/bioinformatics/btag608.

[12] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, p. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[13] V. N. Ioannidis et al., "DRKG – Drug Repurposing Knowledge Graph for COVID-19," 2020. [Online]. Available: https://github.com/gnn4dr/DRKG

[14] S. Bonner et al., "Understanding the performance of knowledge graph embeddings in drug discovery," Artificial Intelligence in the Life Sciences, vol. 2, p. 100036, 2022, doi: 10.1016/j.ailsci.2022.100036.

[15] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in Advances in Neural Information Processing Systems (NeurIPS), 2013.

[16] B. Yang, W.-t. Yih, X. He, J. Gao and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," in Proc. International Conference on Learning Representations (ICLR), 2015.

[17] Z. Sun, Z.-H. Deng, J.-Y. Nie and J. Tang, "RotatE: Knowledge graph embedding by relational rotation in complex space," in Proc. International Conference on Learning Representations (ICLR), 2019.

[18] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov and M. Welling, "Modeling relational data with graph convolutional networks," in The Semantic Web (ESWC), Lecture Notes in Computer Science, vol. 10843, pp. 593–607, 2018, doi: 10.1007/978-3-319-93417-4_38.

[19] R. Zhang, D. Hristovski, D. Schutte, A. Kastrin, M. Fiszman and H. Kilicoglu, "Drug repurposing for COVID-19 via knowledge graph completion," Journal of Biomedical Informatics, vol. 115, p. 103696, 2021, doi: 10.1016/j.jbi.2021.103696.

[20] D. Morselli Gysi et al., "Network medicine framework for identifying drug-repurposing opportunities for COVID-19," Proceedings of the National Academy of Sciences, vol. 118, no. 19, p. e2025581118, 2021, doi: 10.1073/pnas.2025581118.

[21] D. N. Sosa, A. Derry, M. Guo, E. Wei, C. Brinton and R. B. Altman, "A literature-based knowledge graph embedding method for identifying drug repurposing opportunities in rare diseases," in Pacific Symposium on Biocomputing, vol. 25, pp. 463–474, 2020, doi: 10.1142/9789811215636_0041.

[22] M. Zitnik, M. Agrawal and J. Leskovec, "Modeling polypharmacy side effects with graph convolutional networks," Bioinformatics, vol. 34, no. 13, pp. i457–i466, 2018, doi: 10.1093/bioinformatics/bty294.

[23] Z. Gao, P. Ding and R. Xu, "KG-Predict: A knowledge graph computational framework for drug repurposing," Journal of Biomedical Informatics, vol. 132, p. 104133, 2022, doi: 10.1016/j.jbi.2022.104133.

[24] D. Bang, S. Lim, S. Lee and S. Kim, "Biomedical knowledge graph learning for drug repurposing by extending guilt-by-association to multiple layers," Nature Communications, vol. 14, p. 3570, 2023, doi: 10.1038/s41467-023-39301-y.

[25] M. Ali et al., "Bringing light into the dark: A large-scale evaluation of knowledge graph embedding models under a unified framework," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 44, no. 12, pp. 8825–8845, 2022, doi: 10.1109/TPAMI.2021.3124805.

[26] K. K. Teru, E. Denis and W. L. Hamilton, "Inductive relation prediction by subgraph reasoning," in Proc. International Conference on Machine Learning (ICML), 2020.

[27] Z. Zhu et al., "A*Net: A scalable path-based reasoning approach for knowledge graphs," in Advances in Neural Information Processing Systems (NeurIPS), 2023, doi: 10.52202/075280-2591.

[28] M. Galkin, X. Yuan, H. Mostafa, J. Tang and Z. Zhu, "Towards foundation models for knowledge graph reasoning," in Proc. International Conference on Learning Representations (ICLR), 2024.

[29] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in Proc. 23rd International Conference on Machine Learning (ICML), pp. 233–240, 2006, doi: 10.1145/1143844.1143874.

[30] G. Corso, L. Cavalleri, D. Beaini, P. Liò and P. Veličković, "Principal neighbourhood aggregation for graph nets," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[31] E. M. Voorhees, "The TREC-8 question answering track report," in Proc. Eighth Text REtrieval Conference (TREC-8), 1999.

[32] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York: Chapman & Hall, 1993.

[33] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945, doi: 10.2307/3001968.

[34] P. A. McCormick, F. Scott, O. Epstein, A. K. Burroughs, P. J. Scheuer and N. McIntyre, "Thalidomide as therapy for primary biliary cirrhosis: a double-blind placebo controlled pilot study," Journal of Hepatology, vol. 21, no. 4, pp. 496–499, 1994, doi: 10.1016/S0168-8278(94)80092-8.

[35] S. Bonanno et al., "Amifampridine phosphate in the treatment of muscle-specific kinase myasthenia gravis: a phase IIb, randomized, double-blind, placebo-controlled, double crossover study," SAGE Open Medicine, vol. 6, p. 2050312118819013, 2018, doi: 10.1177/2050312118819013.

[36] L. Remijn-Nelissen et al., "Efficacy and safety of amifampridine in myasthenia gravis: a randomized, double-blind, placebo-controlled crossover trial," Neurology, vol. 106, no. 8, p. e214715, 2026, doi: 10.1212/WNL.0000000000214715.

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Meaning
AIH	Autoimmune hepatitis
Any@10	Share of diseases with at least one known drug in the top 10
AUPRC	Area under the precision-recall curve
AUROC	Area under the receiver operating characteristic curve
BCE	Binary cross-entropy loss
CSR	Compressed sparse row matrix format
DGL	Deep Graph Library
DRKG	Drug Repurposing Knowledge Graph
Family (of a disease)	The disease together with its disease-disease neighbours
Filtered rank	Rank of a positive after removing the disease's other positives
GNN	Graph neural network
Held-out disease	A disease whose drug links are hidden from training and evaluated
Hits@k	Fraction of a disease's known drugs ranked in the top k
Indication	A drug-disease link meaning the drug is used to treat the disease
KG	Knowledge graph
kNN	k-nearest neighbours
MG	Myasthenia gravis
MRR	Mean reciprocal rank
NBFNet	Neural Bellman-Ford network
PBC	Primary biliary cholangitis
PrimeKG	Precision Medicine Knowledge Graph
RA	Rheumatoid arthritis
SLE	Systemic lupus erythematosus
MS	Multiple sclerosis
SSc	Systemic sclerosis
T4	NVIDIA Tesla T4 GPU (16 GB)
Zero-shot	Predicting for a disease whose treatments were never seen in training

APPENDIX B: USER MANUAL

The code is in the project repository (branch bpp-v3). The following steps reproduce the results.

1. Download PrimeKG kg.csv from Harvard Dataverse (doi:10.7910/DVN/IXA7BM) into datasets/raw/primekg/.
2. Build the compact graph and splits: python -m bpp.build_primekg --kg datasets/raw/primekg/kg.csv.gz --txgnn-dir datasets/raw/txgnn_splits --out datasets/bpp_primekg
3. Run the tests: python -m pytest -q tests/test_bpp.py
4. Run the baselines on a CPU: python -m bpp.run_baselines --data datasets/bpp_primekg --split splits.json --out outputs/bpp/baselines_targets.json
5. Train on Kaggle: upload kaggle/biopathnet-pp-v3.zip as a dataset, paste kaggle/run_kaggle.py into a notebook cell, select GPU T4 x2 and use Save Version, then Save & Run All. To resume, attach the previous session's runs as a second dataset.
6. Copy the downloaded runs to outputs/bpp/kaggle_sessionN/runs/ and run python -m bpp.analyze and python -m bpp.figures to regenerate every table, figure and paper number.
7. For the TxGNN comparison, run kaggle/run_txgnn.py in a separate notebook with Internet enabled.